Algoritmi paraleli si distribuiți

9. Algoritmi pe grafuri (I): BFS, SSSP cu $\Delta-stepping$

+OpenCL

Aplicații

- Determinarea drumului cel mai scurt într-o hartă
  - De la 15 secunde (implementare naivă) la 10 microsecunde

  Bast, H., Funke, S., Sanders, P., & Schultes, D. (2007). Fast routing in road networks with transit nodes. Science, 316(5824), 566-566.
- Internet-ul si WWW
  - WWW poate fi reprezentat ca un graf direcționat
    - Căutarea și parcurgerea (en., crawl): traversarea grafurilor
    - Analiza hiperlegăturilor (en., links) și ordonarea (en., ranking): algoritmii page rank si HITS
    - Clasificarea paginilor/documentelor și clustering
  - Topologia Internet-ului (i.e., rețeaua de routere) are natural structura unui graf.
- Aplicațiile software
  - Dependințele de bibliotecă sunt de obicei un graf direcționat aciclic: sortare topologică

Aplicații (cont.)

- Calcul științific
  - Structuri de date pentru exploatarea eficientă a sparsității
  - Colorarea grafurilor, arbori de acoperire
  - Mulțimi independente, matchings, tehnici de embedding a grafurilor
- Analiza datelor mari (en., *big data*)
  - Seturi de date complexe, simulari pe date mari (petascale), rețele de senzori
  - Provocări: dimensiunea datelor, eterogenitatea, incertitudinea, valorile lipsă, calitatea datelor
  - Astrofizică, bioinformatică, analiza datelor din rețelele sociale

Aplicații (cont.)

- Bioinformatică
  - Studierea interacțiunilor care apar între componentele care alcătuiesc un sistem biologic
  - Modelarea: predicția interacțiunilor noi
  - Matching/clustering: Anotarea funcțională a unor noi proteine
  - Căi/clstering: Identificarea căilor de metabolizare
  - Clustering/centralitate: identificarea unor noi complexe de proteine
- Analiza datelor din rețelele sociale
  - Publicitatea tintită (en., targeted advertising): clustering și centralitate
  - Studierea modului în care se propagă informația

Cercetare care implică algoritmi paraleli pe grafuri

Aplicații	Metode / Probleme	Algoritmi pe grafuri	Arhitecturi
Analiza rețelelor sociale	Determinarea entităților centrale Detectarea comunităților Dinamica rețelei	Parcurgeri	GPUs FPGAs
WWW	Marketing Căutări	Drumuni minime Conectivitate	Servere X86 multicore
Bioinformatică	Genetică/genomică Metabolism	Flux maxim	Arhitecturi masiv multithread
Calcul științific	Partiționarea grafurilor Colorarea Cuplaj	...	Clustere multicore
Inginerie	VLSI CAD Planificarea căilor	...	Clouds

Workflow

- **Input**: Graful (i.e., lista de adicacență, matrice de adiacență)
- **Problema**: Determinați căile/clusterele/partițiile/șabloanele/etc.
- **Kernel-ul de graf**: parcurgere/drum minim/flux maxim/arbore de acoperire/sortare topologică/etc.
- Factori care influențează alegerea algoritmului
  - Sparsitatea grafului (raportul m/n)
  - Natura grafului: static/dinamic, ponderat/neponderat, distribuția gradelor nodurilor, direcționat/nedirecționat
  - Dimensiunea problemei
  - Granularitatea calculului necesar la nivel de nod/muchie
  - Caracteristici specifice domeniului

Milestones

- 1735: Problema podurilor din Königsberg

https://ro.wikipedia.org/wiki/Problema_podurilor_din_K%C3%B6nigsberg

- 1969: Harary, Graph Theory

https://apps.dtic.mil/dtic/tr/fulltext/u2/705364.pdf

- 1972: Tarjan, DFS and linear graph algorithms
- 1975: Reghbati și Corneil, Parallel connected components
- 1982: Misra și Chandy, Distributed graph algorithms
- 1984: Quinn și Deo, Lucrare studiu (en., survey) asupra ”parallel graph algorithms”

http://akademik.ube.ege.edu.tr/~erciyes/UBI602/p319-quinn.pdf

Reprezentarea grafurilor

- Grafuri statice (m si n sunt fixe)
  - Grafuri dense (m ~= n^2): matrice de adiacență
  - Grafuri rare (en., sparse): listă de adiacență
- Grafuri dinamice
  - Reprezentarea depinde de interogarile cele mai frecvente
    - Inderăm/ștergem muchii sau noduri?
    - Se modifică ponderile muchiilor?
    - Cât de des se modifică graful?
    - Interogările (en., queries) se referă la conectivitate, căi, flux?
  - Principiu de proiectare: optimizarea locală.

Reprezentarea distribuită a grafurilor

- ***Replicare completă***: fiecare procesor deține întreg graful.
- ***Partiționarea 1D***: Fiecare procesor deține **n/p** noduri și toate muchiile adiacente acestor noduri.
- Cum putem să creăm **p** partiții de noduri?
  - Algoritmi de partiționare a grafurilor: Criteriul este optimizarea ”conductanței” (numarul de muchii care traversează tăietura / dimensiunea partițiilor)
  - Amestecarea aleatoare a indecșilor nodurilor asigură că

  ”<numarul de muchii> /<processor>” este aproximativ același.

Reprezentarea distribuită a grafurilor

- ***Partiționarea 2D***: Se consideră un grid 2D logic de procesoare pentru o reprezentare cu matrice de adiacență a grafului.
- Atribuim fiecărui procesor o sub-matrice (mai exact, muchiile din sub-matrice).
- Structuri de date necesare in algoritmii (paraleli) pe grafuri: array, list, queue, stack, set, multiset, tree.
  - Implementările care au constrângeri de perormanță folosesc de cele mai multe ori array-uri.

Localizarea (en., locality)

necesar mare de memorie + localizare slabă => degradarea performaței

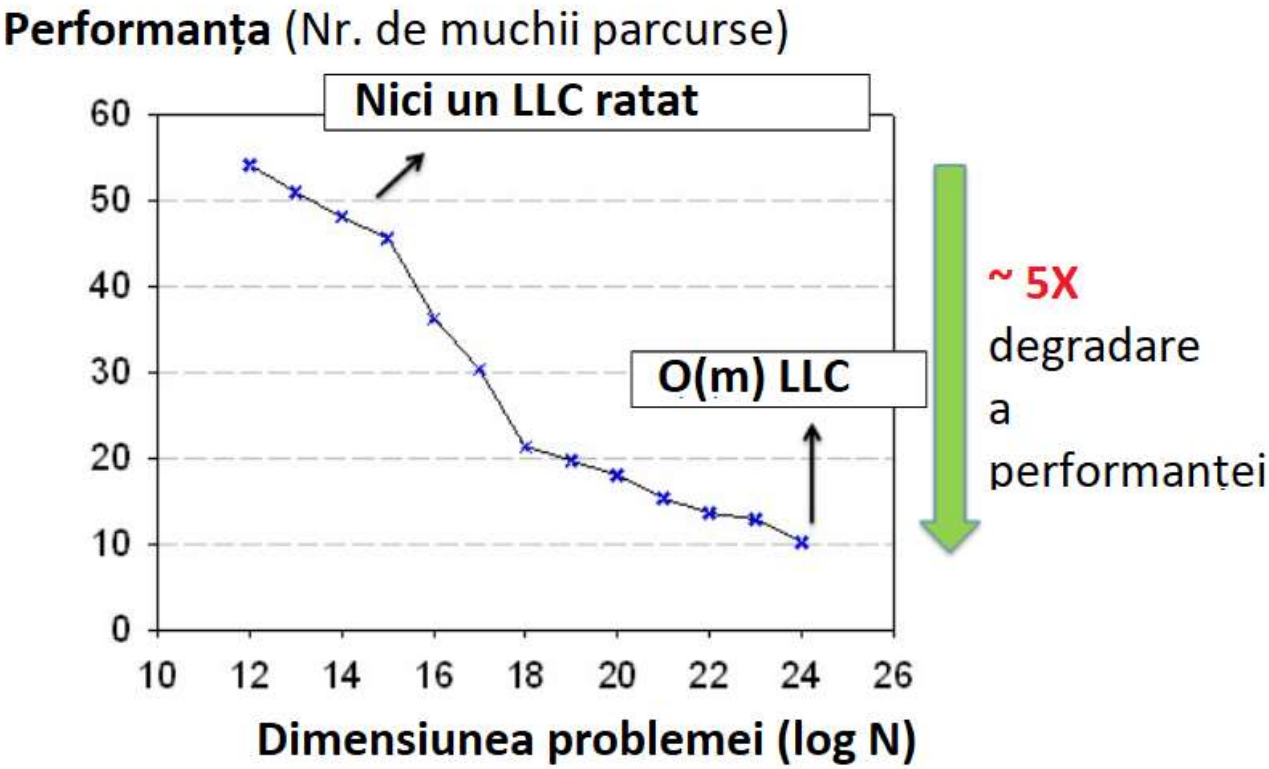
Ex. Problema centralitatii

2.67 Ghz Intel Xeon 5560

(12 GB RAM, 8 MB L3 cache))

Input: Graf sintetic (m=8n)

LLC = Last Level Cache



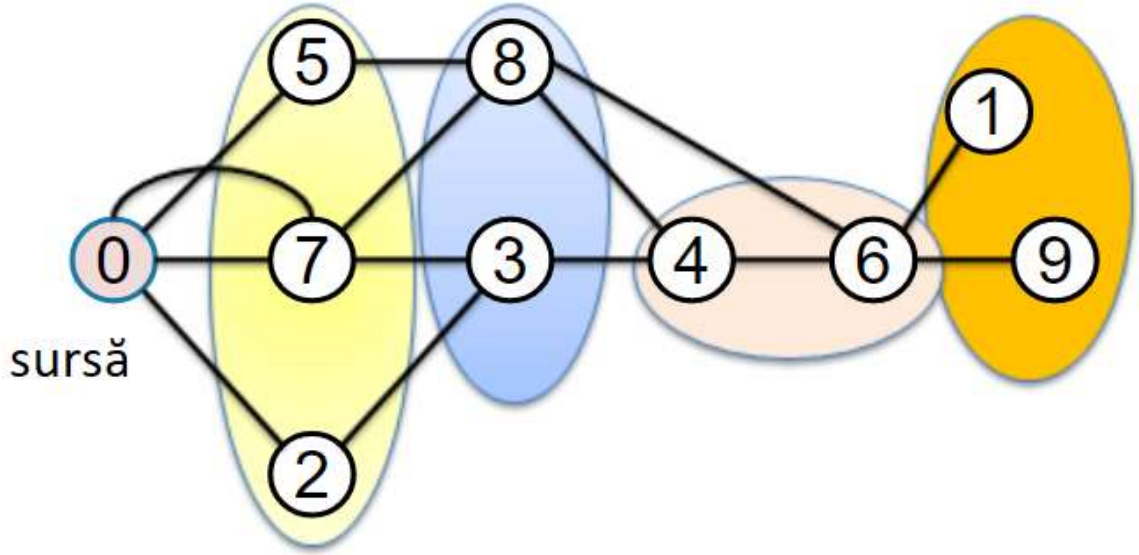
https://images.app.goo.gl/PDH5db3RZkcErC6k9

Scalarea algoritmilor

- Algoritmii clasici pe grafuri nu au imbunătățiri ale performanțelor pe sistemele paralele actuale
  - Topologiile grafurilor folosite de algorimii clasici nu corespund seturilor de date reale.
  - Strategiile actuale de paralelizare se bazează pe îmbunătățirea localizării
  - Algoritmii clasici nu folosesc noile arhitecturi de calcul paralel: ierarhizarea memoriei, eterogenitatea procesoarelor, etc.
  - Adaptarea implementării pentru a minimiza costurile suplimentare de paralelizare este dificilă
    - Memoria partajată: minimizarea costurilor suplimentare ale lacătelor și barierelor.
    - Memorie distribuită: limitarea dimensiunii mesajelor, gruparea convenabilă a mesajelor, suprapunerea/întrețeserea comunicării cu efectuarea calculelor.

BFS paralel

- **Strategia 1**. Frontiera curentă se extinde sincron pentru câte un nivel la un moment dat
  - Abordarea este potrivită pentru grafurile cu diametru mic.
  - Necesită O(D) pași, unde D este diametrul.
  - Nodurile vecine frontierei sunt vizitate în paralel.



BFS paralel

- **Strategia 1. Îmbunătățirea localizării** se poate face prin reetichetarea nodurilor
  - Nodurile inițial nu au etichete, au doar proprietăți: nivel, grad, etc.
  - Lipsa etichetei face problema dificilă (i.e., similar învățării nesupervizate din machine learning).
  - Reetichetarea se poate face prin:
    - Clustering, daca stim numarul de clustere
    - Invățare semisupervizată (topic avansat)
  - Daca gradul nodului este mic atunci adiacențele sunt administrate/stocate explicit într-un index.
  - Nodurile cu grad mare sunt procesate în ordine dar si prin blocarea în cache
    - Scopul este de a forma blocuri dense în jurul nodurilor cu grad mare.

BFS paralel

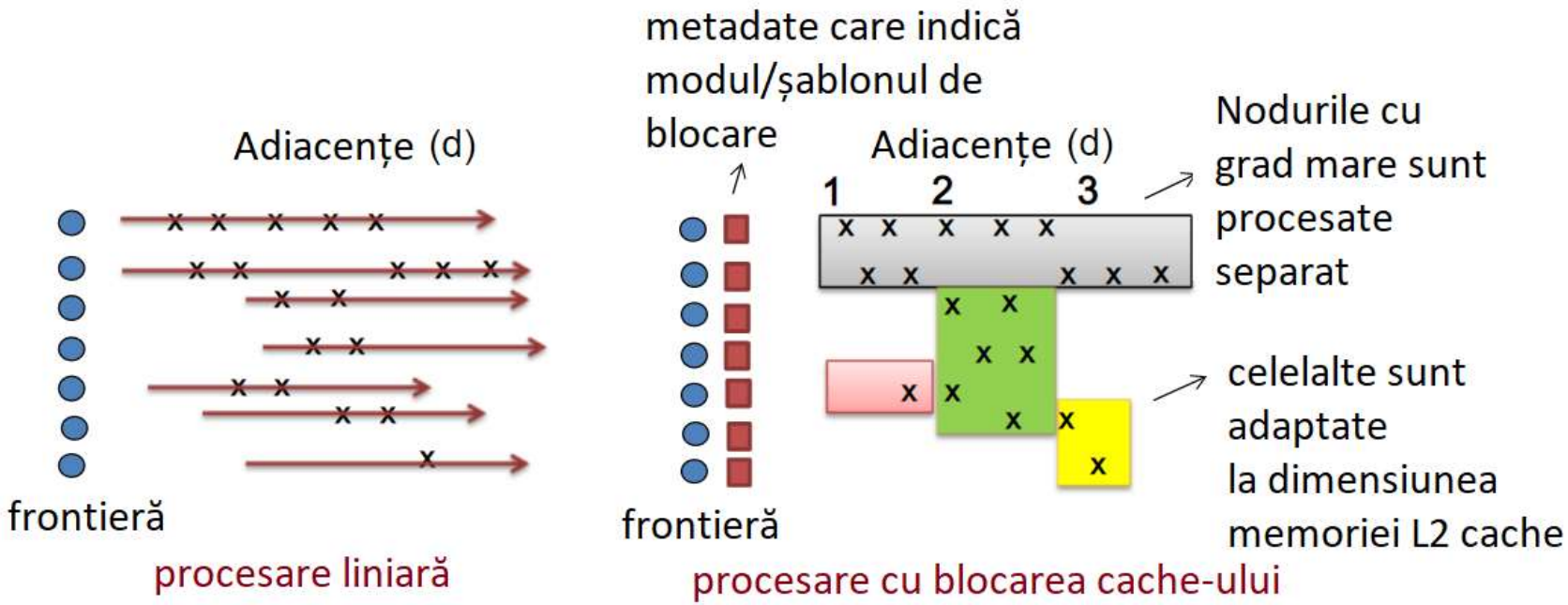
- **Strategia 1. Optimizări**. Scopul este să valorificăm cache-ul partajat și să diminuăm lipsa informațiilor necesare din cache-ul privat sau TLB (en., *translation lookaside buffer*)

  http://www.cs.cmu.edu/afs/cs.cmu.edu/user/tcm/www/thesis/subsubsection2_10_1_2_1.html

  - Sortăm listele de adiacență ale fiecărui vector, astfel accesele la memorie vor fi ordonate și se va fi diminuată lipsa informațiilor din TLB.
  - Permutăm etichetele nodurilor pentru îmbunătățirea localizării spațiale
  - Blocăm in cache (en., cache blocking) muchiile vizitate recent pentru a exploata localizarea temporală.

BFS paralel

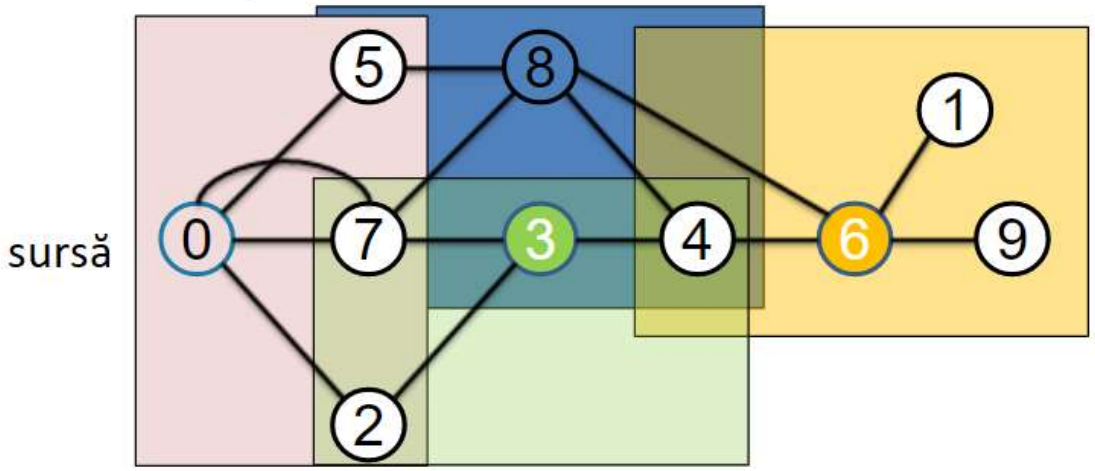
- Îmbunătățirea localizării prin blocarea cache-ului



BFS paralel

- **Strategia 2**. Îmbină mai multe traversări concurente[1]
  - Abordarea este potrivită pentru grafuri cu diametru mare
  - Parcurgerea muchiilor/drumurilor este gestionată/limitată de *super-noduri*
  - Abordarea este asemanatoare cu APSP între *super-noduri*

[1]. Ullmann, https://apps.dtic.mil/sti/pdfs/ADA250894.pdf



BFS: Optimizări specifice arhitecturii HW

- *Software prefetching* pe procesoarele Intel Core i7
  - Încărcarea speculativă a indecșilor și a nodurilor vecine frontierei in cache va reduce cantitatea de date lipsă (en., cache miss) necesară procesării.
  - Alinierea listelor de adiacență (i.e., localizarea spațială) optimizează accesul la memorie prin reducerea fragmentării.
  - *Hugepage support* contribuie la reducerea semnificativă a lipsei infomatiilor necesare din TLB

  https://en.wikipedia.org/wiki/Page_(computer_memory)
  - Alocarea în memoria NUMA-aware este avantajoasă pentru politica *first-touch*

  https://queue.acm.org/detail.cfm?id=2513149

Algoritmi SSSP (en., single source shortest paths)

- Abordări
  - **Cozi de prioritate paralele:**

    G.S. Brodal, J.L. Tr¨aff, and C.D. Zaroliagis. A parallel priority queue with constant time operations. Journal of Parallel and Distributed Computing, 49(1):4–21, 1998.

    J.R. Driscoll, H.N. Gabow, R. Shrairman, and R.E. Tarjan. Relaxed heaps: An alternative to fibonacci

    heaps with applications to parallel computation. Communications of the ACM, 31(11):1343–1354, 1988.
  - **Randomizat:**

    J. Ullman and M. Yannakakis. High-probability parallel transitive closure algorithms. In Proc. 2nd Ann. Symp. Parallel Algorithms and Architectures (SPAA-90), pages 200–209, Crete, Greece, July 1990. ACM.
  - **<u>Algoritmul $\Delta - stepping$</u>**
  - **Implementări cu memorie distribuită bazate pe partiționarea grafului**
  - **Euristici pentru balansarea încărcării**

Algoritmul $\Delta - stepping$

- Context SSSP (en., Single Source Shortest Paths, Dijkstra)
  - **G (n, m)** este un graf direcționat cu ***n*** noduri si ***m*** muchii
    - ***s*** este nodul sursă
    - ***c*** este o funcție care atribuie ponderi nenegative reale tuturor muchiilor din G.
  - **Scop**: determinarea costurilor minime ale drumurilor de la ***s*** la toate nodurile ***v*** în care se poate ajunge din ***s***.
  - **Soluție secventială**: Algoritmul lui Dijkstra, O(n logn +m) cu movile.
    - Bellman-Ford: permite muchii de cost negativ, consideră nodurile în paralel, dar este ineficient (O(nm)) față de Dijkstra.
  - **Abordare paralelă**: Algoritmul $\Delta - stepping$
    - Meyer, U., & Sanders, P. (1998, August). δ-stepping: A parallel single source shortest path algorithm. In *European symposium on algorithms* (pp. 393-404). Springer, Berlin, Heidelberg. https://www.cs.utexas.edu/~pingali/CS395T/2012sp/papers/delta-stepping.pdf
    - Meyer, U., & Sanders, P. (2003). Δ-stepping: a parallelizable shortest path algorithm. *Journal of Algorithms*, *49*(1), 114-152.
    - https://www.sciencedirect.com/science/article/pii/S0196677403000762

Algoritmul $\Delta - stepping$

- Notații și constrângeri
  - **d** = gradul maxim din graf
  - **L** = costul celui mai lung drum din graf
    - Costul unui drum este egal cu suma costurilor muchiilor care fac parte din acel drum
    - Lungimea unui drum este egala cu numarul de muchii parcurse
  - **d/n** probabilitatea ca între două noduri să avem muchie (en., edge probability)
  - Modelul G(n, p), unde p=d/n
    - Pentru d<1, fiecare componentă conexă este mică
    - Pentru d>1, există o componentă conexă mare care conține un anumit procent din noduri. Cu cât d este mai mare cu atât componenta are mai multe noduri.
  - Ex. Nodes = persoane, edge = relatia de prietenie

  p = d/n este probabilitatea ca două persoane să se întâlnească și să devină prietene. Această valoare este statictic independentă de toate celelalte relații de prietenie existente.

  d = numărul de prieteni pe care poate să-l aibă cineva.

  Cunoscând valorile lui d și n, cât de mari sunt componentele conexe?

Algoritmul $\Delta - stepping$

- Costurile minime ale nodurilor se pot calcula prin:
  - *Determinare/cucerire*: ca la Dijkstra => O(n) pași, deoarece un nod este cucerit o singură dată.
  - *Corectare*: ca la Bellman-Ford: costurile nodurilor sunt fixate temporar, ele putând fi modificate ulterior prin alte relaxări.
- Abordare generală a Algoritmului $\Delta - stepping$
  - Nodurile eligibile (cu distanțele lor preliminare) se administrează in array-ul B de bucket-uri de dimensiune/lățime Δ, care este un număr real pozitiv.
  - Fiecare bucket ***i*** conține muchii de cu costuri între [i*Δ, (i+1)*Δ]
  - Dacă L = cel mai lung drum dintre cele mai scurte drumuri de la s la orice v
    - Atunci array-ul B conține $\lceil L/\Delta \rceil$ bucket-uri
    - Prin reutilizare bucket-urilor goale sunt suficiente $b = \max_{e\epsilon} \lceil c(e)/\Delta \rceil + 1$

Algoritmul $\Delta - stepping$

- ***Distanțele preliminare*** (i.e., ***tent(v)***) sunt inițial ∞ sau ponderea unui drum de la s la v. Acestea sunt îmbunătățite (i.e., micșorate) prin relaxarea muchiilor (v, w)∈ E

  tent(w) = min {tent(w), tent(v) + c(c, w)}
- În timpul fiecărei faze, muchiile cu cost ≤ Δ dintr-un bucket (muchii *ușoare*) sunt relaxate. Relaxarea poate avea ca rezultat adăugarea de noi noduri în bucket care vor fi analizate/șterse în faza următoare.
  - Nodurile șterse pot fi reinserate intr-o fază ulterioară dacă distanța preliminară se îmbunătățește (i.e., se micșorează). Aceasta este *corectarea*.
- Toate muchiile (din toate bucket-urile) cu cost > Δ sunt relaxate după ce ponderile preliminare ale nodurilor sursă ***v*** au fost stabilite.
  - Acest lucru se întamplă atunci când toate bucket-urile nu mai conțin muchii *ușoare*.
- Varianta secvențială a algoritmului caută bucket-uri care nu sunt goale.

Algoritmul $\Delta - stepping$

- Ștergerea și relaxarea unui nod dintr-un bucket se poate realiza în paralel și în orice ordine atâta timp cât relaxarea este implementată ca o operație atomică (i.e., relaxările pentru același sunt efectuate secvențial).
  - Ca abordare alternativă, toate relaxările unui nod se pot combina într-o singură relaxare a nodului cu cea mai mică distanță.

Algoritmul $\Delta - stepping$

- Algoritm bazat pe corectarea distanțelor
  - Poate relaxa muchii și din nodurile necucerite
    - Valoarea inițială a distanței este fixată ca în Dijkstra
    - Valoarea distanței este modificată/updatată/corectată ca în Bellman-Ford
- $\Delta - stepping$: este o implementare cu bucket-uri aproximative a algoritmului lui Dijkstra
- Nodurile sunt organizate/împărțite în bucket-uri în funcție de Δ
- Folosirea bucket-urilor permite mai multe vizitări (în paralel!) ale nodurilor în funcție de distanțele curente.
  - Obs: Algoritmul Dijkstra vizitează o singură dată fiecare nod!

Algoritmul $\Delta - stepping$ (cont.)

- Bucket-urile de noduri sunt create în funcție de distanța curentă față de nodul sursă
- B[i] conține nodurile care au distanța curentă în [i*Δ, (i+1)*Δ]
- Bucket-urile goale pot fi refolosite pentru a economisi spațiu
- Bucla/ciclul exterior/principal parcurge bucket-urile
- Bucla/ciclul interior procesează bucket-ul până când acesta devine gol

Algoritmul $\Delta - stepping$ (cont.)

- Procesarea unui bucket
  - Fiecare nod din bucket are muchii care ies. Acestea sunt fie:
    - ”ușoare”, pentru w ≤ Δ sau
    - ”grele”, pentru w > Δ
  - Toate muchiile ”ușoare” sunt relaxate (ca în Dijkstra)
  - Relaxarea unei muchii poate avea ca rezultat adăugarea nodului destinație în bucket-ul curent
  - Bucket-ul curent este procesat până cand se golește, apoi sunt relaxate muchiile ”grele”

Algoritmul $\Delta - stepping$ (cont.)

- Alegerea lui Δ (număr real pozitiv, reprezintă lățimea bucket-ului)
  - Δ=1 și costuri întregi reduce algoritmul la Dijkstra (fiecare nod este in bucket-ul său). Vom avea L bucket-uri și nici o reinserare.
  - Δ ≥ n * *max_weight* reduce algoritmul la Bellman-Ford (adică utilizarea exclusivă a primului/singurului bucket)
    - *max_weight* este ponderea maximă din graf.
    - Sunt necesare k parcurgeri ale bucket-ului, k = adâncimea maximă a arborelui de căi minime. Timpul necesar devine **O(k*m)**.
  - $\Delta - stepping$ are ca scop determinarea facilă a unei valori convenabile pentru Δ care să conducă la un compromis rezonabil între cele două extreme.
  - Δ mai mic (i.e., putine bucket-uri) => multe reconsiderări ale nodurilor
  - Δ mai mare (i.e., multe bucket-uri) => multe traversări ale bucket-urilor

Algoritmul $\Delta - stepping$ (cont.)

- Costurile suplimentare sunt date de:
  - Reinserări: inserări ale nodurilor care au fost anterior șterse
  - Re-relaxări: ale muchiilor care ies din nodurile reinserate
- **Lema 1.** Numărul total de reinserări este maxim $n_\Delta = |C_\Delta|$ și numărul maxim de re-relaxări este maxim $m_\Delta = |C_{\Delta+}|$.
  - Unde:
    - ***Δ-path*** este un drum de lungime maximă Δ în care o muchie apare o singură dată.
    - C_Δ este multimea de perechi/muchii (u, v) care se află pe un anumit *Δ-path*
    - $n_\Delta = |C_\Delta|$ este numărul de muchii din *Δ-path*
    - $C_{\Delta+}$= mulțimea de triplete <u, v', v> astfel încât (u, v') ϵ C_Δ și (v', v) este muchie *ușoară*
  - Timpul secvențial cu acești parametri este

  O(n + m+ n_Δ + m_Δ + L/Δ), care pentru ponderi aleatoare face ca n_Δ + m_Δ=O(n+m) cu mare probabilitate pentru $\triangle = \Theta(1/d)$.

Algoritmul $\Delta - stepping$ (cont.)

- Alegând $\triangle = \Theta(1/d)$ pentru grafuri cu:
  - Gradul maxim d
  - Costurile muchiilor sunt aleatoare si normal distribuite în [0,1]

Algoritmul secvențial are nevoie de **O(n + m + d*L)** timp mediu pentru rezolvarea SSSP.

- Algoritmul, face împarte muchiile dintr-un bucket în:
  - ”***ușoare***”, cu costul ≤ Δ
  - ”***grele***”, cu costul > Δ
- Obs: Alegerea lui $\triangle$ determină:
  - Numărul de bucket-uri
  - Modul în care se împart muchiile în *ușoare* sau *grele* în cadrul unui bucket.

Algoritmul $\Delta - stepping$ (cont.)

- Parelalismul se obține prin scoaterea concurentă a tuturor nodurilor din primul buchet care nu este gol (i.e., bucket-ul curent) și relaxarea muchiilor ușoare într-un singură fază.
- Dacă un nod ***v*** este scos din bucket-ul B[i] (costul determinat la acest moment NU este final!) acesta poate fi analizat într-o fază ulterioară, atunci cand poate fi reinserat în bucket-ul B[i], iar muchiile *u*șoare care ies din ***v*** vor fi relaxate încă o dată.
- Muchiile *grele* rămase (care ies din toate nodurile scoase până acum din B[i]) sunt relaxate o singură dată atunci când B[i] se golește.
- Scopul algoritmului este să caute bucket-uri care mai au muchii ușoare și să le relaxeze în paralel. La final, toate muchiile grele sunt relaxate într-o singură fază.

Algoritmul $\Delta - stepping$ (cont.)

Lema 2. Pentru orice valoare a lui Δ numărul de faze este maxim $\frac{L}{\Delta} l_{\Delta}$.

Teorema 1. Algoritmul secvențial $\Delta - stepping$ poate rezolva problema SSSP în $O(n + m + L/\Delta + n_{\Delta} + m_{\Delta})$.

- Dacă $L/\Delta + n_{\Delta} + m_{\Delta} = O(n+m)$ atunci algoritmul este liniar in raport cu n și m.

Pentru varianta paralelă, scopul este determinarea valorii optime pentru Δ : suficient de mică pentru a obține eficiență, dar suficient de mare pentru avea paralelism.

Lema 3. Pentru $\Delta = O(1/d)$, nu există nici un Δ-path cu mai mult de $l_{\Delta} = O(\log n / \log\log n)$ muchii, cu mare probabilitate.

Algoritmul $\Delta - stepping$ [pseudocod]

foreach v ∈ V **do** -- *initializare noduri*

heavy(v) := {(v, w) ∈ E : c(v, w) > Δ } -- *determinăm muchiile "grele"*

light (v) := {(v; w) ∈ E : c(v; w) ≤ Δ -- *determinăm muchiile "usoare"*

tent (v) := ∞ -- *nodurile nevizitate*

relax (s, 0); i := 0 -- *nodul sursă este la distanță 0*

while (! isEmpty(B)) **do** -- *cat timp mai avem noduri în coadă*

S := ∅ -- *nici un nod nu a fost încă șters din acest bucket*

while B[i] != ∅ **do** -- *începem o fază nouă*

Req := {(w, tent(v) + c(v, w)) : v ∈ B[i] ∧ (v, w) ∈ light(v)}

S := S ∪ B[i]; B[i] := ∅ -- *memorăm nodurile șterse*

foreach (v, x) ∈ Req **do** relax(v, x) -- putem reinsera noduri

Req := {(w, tent(v) + c(v, w)) : v ∈ S ∧ (v, w) ∈ heavy(v)}

foreach (v, x) ∈ Req **do relax**(v, x) -- relaxăm muchiile amânate anterior

i := i + 1 -- trecem la următorul bucket

procedura **relax**(v, x) -- *drum mai scurt la v?*

if x < tent(v) **then**

-- *stergem pe v din bucket*

B[tent(v)/delta] := B[tent(v)/delta] \ {v}

-- *Inseram în noul bucket*

B[x/delta] := B[x/delta]∪ {v}

tent(v) := x

Algoritmul $\Delta - stepping$ (cont.)

Performanța depinde de alegerea valorii lui Δ în funcție de caracteristicile grafului.

Analiza complexității [varianta secvențială]

- Pentru ponderi aleatoare uniform distribuite în [0,1] costurile suplimentare pentru re-inserări și re-relaxări sunt maxim O(n+m) pentru $\triangle$=Θ(1/d) .
- Dacă **L** este costul maxim al celui mai scurt drum atunci timpul mediu devine O(n + m + d*L)
  - Pentru d*L = O(n+m) timpul mediu este liniar in raport cu n si m, adică O(n+m).

  Obs: Acesta este cazul grafurilor direcționate cu grad cunoscut și limitat.

Algoritmul $\Delta - stepping$ (cont.)

Analiza complexității [varianta paralelă]

- Paralelizarea necesită o limitarea a numărului de faze.
  - Numărul de faze este maxim 1 + numărul maxim de muchii care apar în orice Δ-***cale***.
  - l_Δ este 1+ numărul maxim de muchii necesare pentru a conecta orice pereche de noduri (u, v) printr-un drum de cost minim.

$$l_\Delta = 1 + \max_{(u,v)\in C_\Delta} \min\{|A|: A = (u, \dots, v)\ este\ un\ drum\ de\ cost\ minim\}$$

- Paralelizarea simplă are complexitatea O(L/Δ * d * l_Δ * log n), ***<u>cu mare probabilitate</u>***.
  - În contextul în care $l_\Delta = O\left(\frac{\log n}{\log\log n}\right)$ se întâmplă cu mare probabilitate pentru $\triangle$=Θ(1/d).
  - *Cu mare probabilitate înseamnă că probabilitatea unui eveniment este cel puțin* $1 - n^{-\beta}$ *pentru price constantă* $\beta > 0$.
  - *d poate fi anulat cu ajutorul unor algoritmi avansati de echilibrare a încărcării*

Algoritmul $\Delta - stepping$ (cont.)

Exemplu de rulare intuitiv:
https://cs.iupui.edu/~fgsong/LearnHPC/sssp/deltaStep.html

Neo4j Graph Data Science library

Această bibliotecă oferă implementări eficiente pentru versiunile paralele ale celor mai întâlniți algoritmi pe grafuri pentru Neo4j.

https://neo4j.com/docs/graph-data-science/current/alpha-algorithms/single-source-shortest-path/

" We implement a delta-stepping algorithm that has been shown to outperform Dijkstra's."

Kranjčević, M., Palossi, D. and Pintarelli, S., 2016. Parallel delta-stepping algorithm for shared memory architectures. *arXiv preprint arXiv:1604.02113*.

https://arxiv.org/pdf/1604.02113v1.pdf

Referințe

- Madduri, K., Bader, D. A., Berry, J. W., & Crobak, J. R. (2007, January). An experimental study of a parallel shortest path algorithm for solving large-scale graph instances. In 2007 Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments (ALENEX) (pp. 23-35). Society for Industrial and Applied Mathematics.
- Meyer, Ulrich, and Peter Sanders. "Δ-stepping: a parallelizable shortest path algorithm." Journal of Algorithms 49, no. 1 (2003): 114-152.
- https://graph500.org/?page_id=12#sec-7

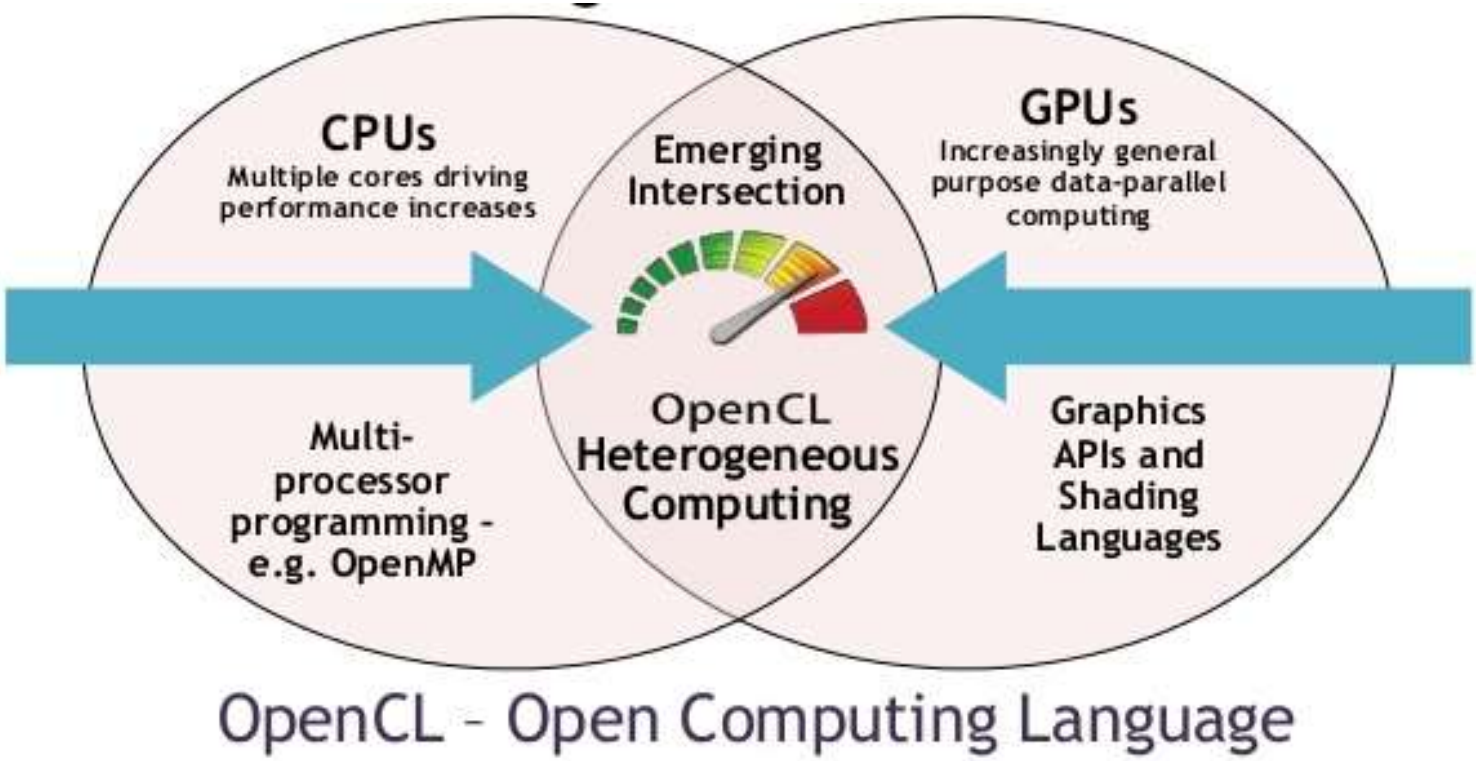
OpenCL (Open Computing Language)

- https://developer.nvidia.com/opencl
- Este un API de nivel jos pentru calcul eterogen care rulează pe GPU cu CUDA.
- Dezvoltatorii pot crea și rula kernel-uri folosind limbajul C pe GPU.
- Este o marcă înregistrată a Apple Inc. folosită sub licență de Khronos.

https://streamhpc.com/blog/2017-05-04/what-is-khronos-as-of-today/

  - Apple a făcut propunerea inițială și este editor al specificațiilor

Standard pentru programarea pe platforme eterogene (CPUs, GPUs, etc.)



Scopuri ale OpenCL

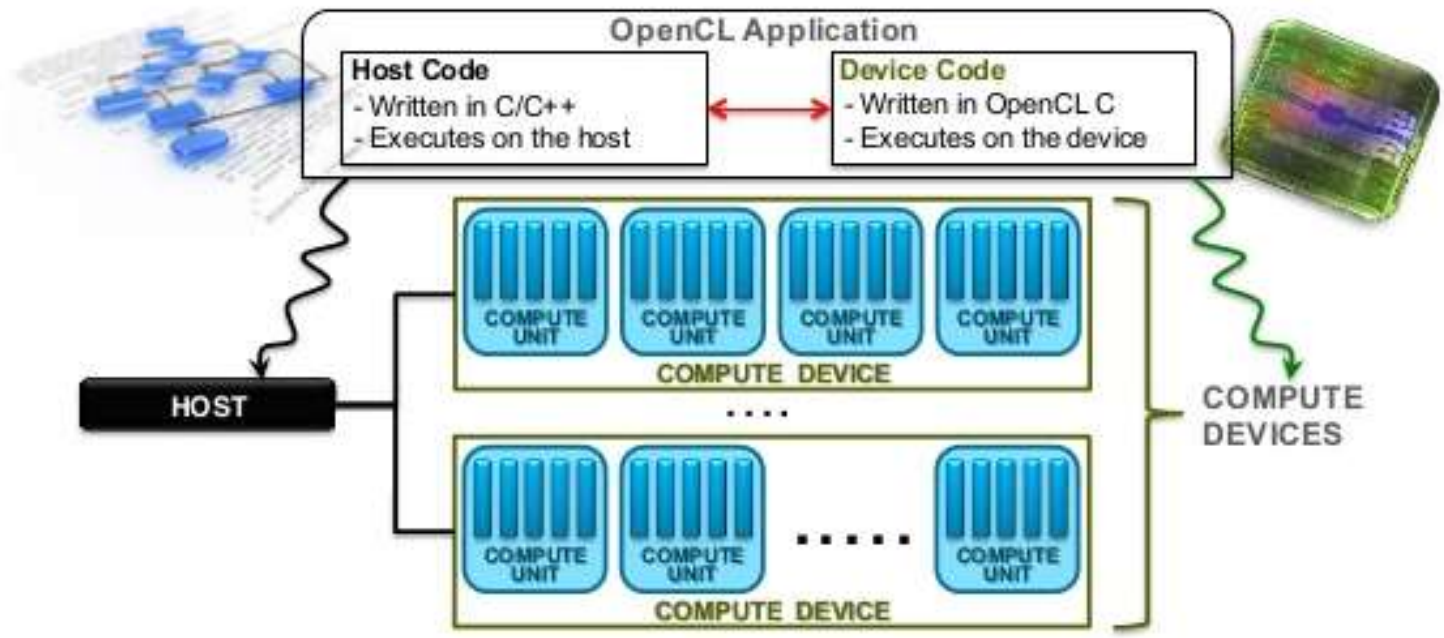
- Să permită utilzarea mai multor resurse de calcul dintr-un sistem
  - CPUs, GPUs, alte procesoare/acceleratoare

  Folosind OpenCL programatorii scriu un singur progrtam care este portabil si care folosește TOATE resursele dintr-o platformă eterogenă.
- Model de calcul paralel eficient
  - Kernele dezvoltate în limbajul ANSI C99
- Abstractizare de nivel jos
  - Obținerea de performanță pentru o gamă largă de dispozitive
- Interoperabilitate cu API-urile grafice
  - Suport pentru OpenGL, OpenGL ES și DirectX

Modelul platformei OpenCL

- O gazdă este conectată la unul sau mai multe dispozitive (i.e., CPU, GPU, sau alte procesoare/acceleratoare)
- Fiecare dispozitiv este alcătuit din una sau mai multe unități de calcul (i.e., un core, multi-procesor, etc.)
- Fiecare unitate de calcul este împărțită în elemente de procesare, acestea execută codul ca SIMD.

Anatomia unei aplicații OpenCL



Gazda trimite comenzi către dispozitive: 1) pentru a transfera date între memoria gazdei li memoria dispozitivelor si 2) pentru a executa cod pe dispozitiv(e).

Anatomia unei aplicații OpenCL (cont.)

- Codul serial este executat pe un singur fir al sistemului gazdă (CPU)
- Codul paralel este executat pe mai multe fire pe dispozitive (GPU)

Modelul de execuție OpenCL:

- Aplicațiile rulează pe sistemul gazdă care trimite cereri către dispozitive în vederea execuției de sarcini (en., work).
  - Sarcina (en., work item) = ceea ce trebuie executat pe dispozitiv
  - Kernel = codul C care trebuie executat în cadrul sarcinii.
  - Program = colecție de kernele (si eventual alte functii) administrare de sistemul gazdă
  - Context = mediul în care se execută sarcinile: dispozitivele cu memoria lor și coada de comenzi.
  - Coada de comenzi = O coadă folosită de aplicația gazdă pentru a trimite sarcini unui dispozitiv.

Execuția kernel-urilor

- O sarcină (en., work-item) este executată de un element de calcul (en., compute element)
- Fiecare grup de sarcini (en., work-group) este executat pe o unitate de calcul (en., compute unit).
  - Mai multe grupuri de sarcini concurente se pot găsi pe o unitate de calcul în funcție de necesarul de memorie și de resursele unității de calcul.
- Fiecare kernel este executat pe un dispozitiv (en., compute device)

Beneficiile utilizării grupurilor de sarcini (en., work-group)

- Scalabilitatea automată peste dispozitive cu numar diferit de unități de calcul
  - Grupurile de sarcini se pot executa în orice ordine, concurent sau secvențial.
- Cooperare eficientă între sarcininile din cadrul aceluiași grup de sarcini
  - Memorie partajată rapidă și sincronizată
- Independența dintre grupurile de sarcini oferă scalabilitate
  - Un kernel se poate scala/rula pe orice număr de unități de calcul.

Memoria în OpenCL

- Spațiul de adrese este:
  - Privat: Acces R/W numai la nivel de sarcina (i.e., work-item).
  - Local: Acces R/W numai la nivel de grup de sarcini (en., work-group) .
  - Global/constant: vizibil către toate grupurile de sarcini.
  - Al gazdei: accesibil de CPU.

- Sincronizarea
  - Toate sincronizările necesare pentru acces la memorie trebuie realizate explicit.

  Obs: Administrarea memoriei se face explicit: Datele trebuie mutate pe ruta gazdă->global->local … și înapoi.

Dezvoltarea de aplicații cu OpenCL

- Limbajul OpenCL si API necesar
  - API la nivel de platformă (apelabil din sistemul gazdă)
    - Abstractizarea accesului la resursele de calcul
    - Interogarea, selectarea si inițializarea dispozitivelor
    - Crearea de contexte de calcu și a cozilor de lucru
  - API la rulare (apelat de gazdă)
    - Executa kernel-uri
    - Stabilirea configurației de execuție a kernel-ului
    - Administrarea planificarilor, calculelor și a resurselor de memorie
  - Limbajul OpenCL
    - Kernel-urile sunt scrise în C
    - Sit disponibile multe funcții
    - Compilarea se poate face JIT/online sau offline

Limbajul OpenCL

- Calificarea funcțiilor
  - Calificatorul __kernel declara o funcție ca și kernel.
- Calificarea spatiului de momorie
  - __global, __local, __constant, __private
- Funcții pentru lucrul cu sarcinile
  - get_work_dim(), get_global_id(), get_local_size()
- Funcții pentru lucrul cu imaginile
  - Imaginile pot fi accesate prin funcții built-in
- Funcții pentru sincronizare
  - Barierele: toate sarcinile dintr-un grup trebuie să execute funcția barieră înainte ca orice sarcină din grup să poată fi executată.

Tipuri de obiecte în OpenCL

- cl_platform_id: identificator pentru o anumită platformă
- cl_device_id: identificator pentru un anumit dispozitiv
- cl_context: *handle* pentru un context de calcul
- cl_command_queue: *handle* pentru o coadă de comenzi (a unui dispozitiv)
- cl_mem: *handle* pentru o resursă de memorie (dintr-un context)
- cl_program: *handle* pentru un program (într-o bibliotecă de kernel-e)
- cl_kernel: *handle* pentru un kernel.
- Toate obiectele au numarul de referințe numărat, iar sistemul are propriul garbage collector
  - Când numărul de referințe ajunge la 0 atunci memoria obiectului este eliberată.

OpenCL C

- Derivat din ISO C99
- Proprietăși adăugate limbajului
  - Existența sarcinilor si a grupurilor de sarcini
  - Tipul vector
  - Sincronizare
- Functii build-in pentru
  - Lucrul cu imagini
  - Lucrul cu sarcini
  - Funcții matematice

Limitări:

- Nu sunt permise apelurile recursive
- Nu sunt permiși pointerii la funcții
- Sunt permisi pointerii la pointeri dar intr-un kernel, nu ca parametri.
- Nu sunt permise array-uri sau structuri de dimensiune variabilă.
- Etc..